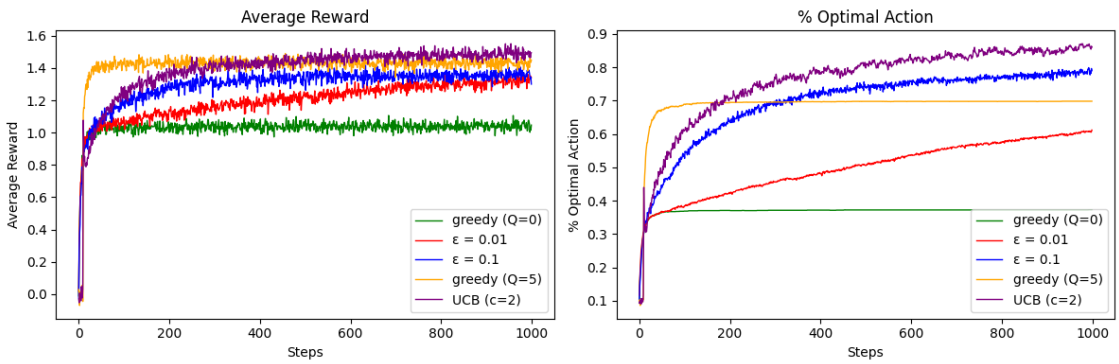
Reinfocement Learning Assignment 1

Mateusz Susmarski (14458632)

1. The following figure shows the average reward and the percentage of optimal actions over time for several action-selection methods: ε-greedy with $Q = 0$ (using $\varepsilon = 0.01$ an $\varepsilon = 0.1$), greedy optimisic initialization ($Q_0 = 5$), and the upper-confidence-bound (UCB) method with $c = 2$ (arbitrary parameter choice for now):



These results we obtained using the following action-selection methods:

- *Greedy method*:

$$A_t = \arg\max_{a \in \mathcal{A}} Q_t(a)$$

- *ε-greedy method*:

$$A_t = \begin{cases} \arg\max_{a \in \mathcal{A}} Q_t(a), & \text{with probability } 1 - \varepsilon \\ \text{a random action } a \in \mathcal{A}, & \text{with probability } \varepsilon \end{cases}$$

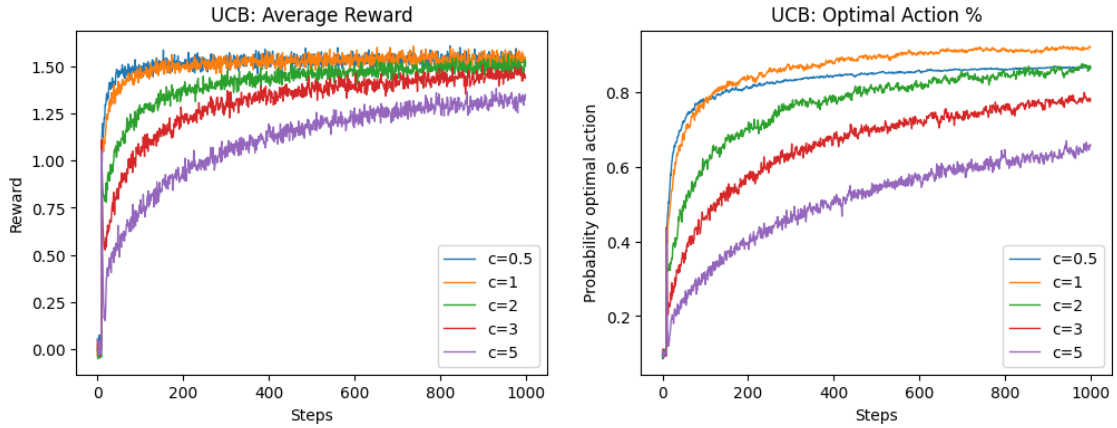
- *Upper Confidence Bound (UCB)*:

$$A_t = \arg\max_{a \in \mathcal{A}} \left[Q_t(a) + c\sqrt{\frac{\log t}{N_t(a)}} \right]$$

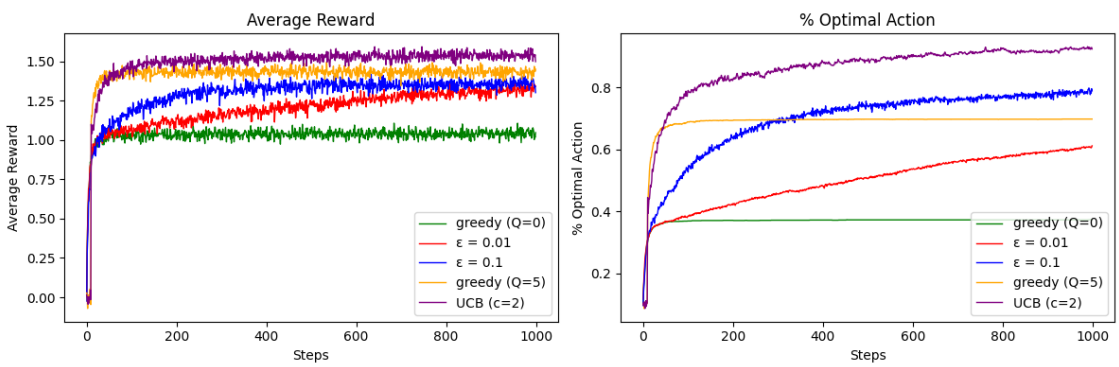
For the UCB method, the exploration parameter, c, was initially set arbitrarily (the same is in the book). To determine a good value, I evaluated the performance of UCB across a range of c values by comparing both the average reward and the percentage of optimal actions over 1000 time steps. I have considered the following set of parameter values:

$$c = \{0.5, 1, 2, 3, 5\}$$

The resulting plots are:



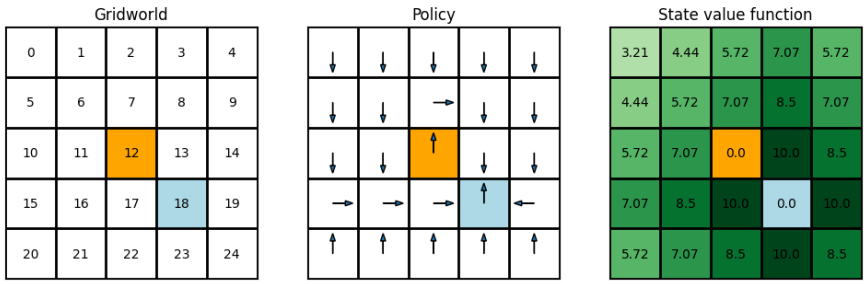
We observe that both $c = 1$ and $c = 2$ consistently perform better than other values, giving us the highest average rewards and the largest fraction of optimal action selections. Although $c = 0.5$ achieves a relatively high average over time, it obtains a lower fraction of optimal action selections compared to $c = 1$ as time goes on. By comparing these trade-off, between average reward and optimal action selection, I have selected $c = 1$ as the better parameter value. The following figure shows the final comparison of all methods (ε-greedy, greedy, and UCB with selected c):



2. (a) In order to implement the Value Iteration algorithm for the GridWorld environemnt, I have followed the provided pseudocode in the book (Sutton % Barto, page 83). The value function is updated using the Bellman optimality equation:

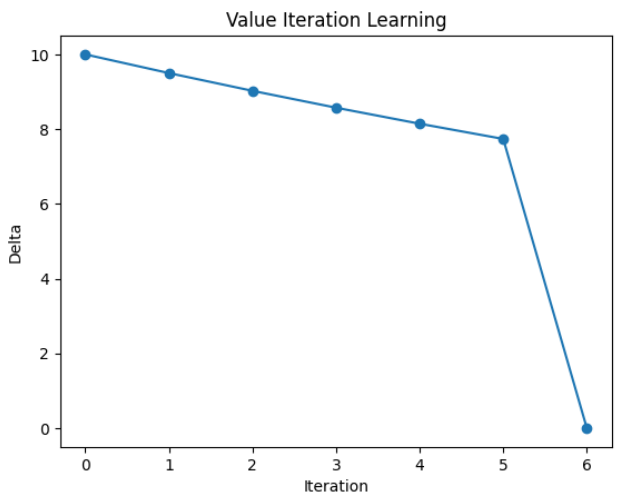
$$V(s) \longleftarrow \max_{a \in \mathcal{A}} Q(s,a), \quad Q(s,a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \Big[r + \gamma V(s') \Big]$$

The value iteration algorithm converged after 7 iterations, and the resulting optimal policy and state-value functions are:



Note: Although policy (arrows) are displayed for both the fire and water states, these should not be interpreted as meaningful actions (as they are terminal). Once the agent reach those states, it remains in the same state regardless of action. I computed all policies for completeness.

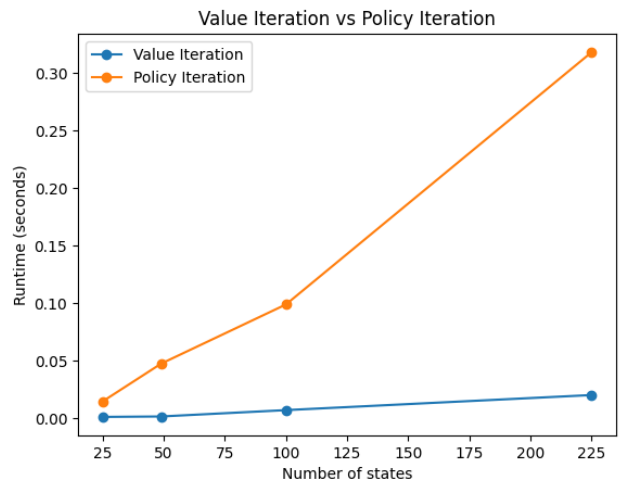
To see wether the algorithm is truly learning, we examine the stability of the value functions over time. We compute $\Delta_t = \max_{s \in \mathcal{S}}\{\Delta_{t-1}, |V_t(s) - V_{t-1}(s)|\}$ (Delta) as a function of iterations:



(b) To compare the speed of Value Iteration and Policy Iteration as a function of state space size, I have considered the following set of dimensions:

$$\text{size} \in \{(5,5), (7,7), (10,10), (15,15)\}$$

For each size, I have measured the runtime required for both algorithms to converge. The results are:



As expected, Value Iteration converges faster then Policy Iteration across all states space sizes.

Note: For this part I have chosen the policy iteration with `argmax = False` and convergence was evaluated based on the state-value functions rather than the policy (Seminar 4, ex. 2e).

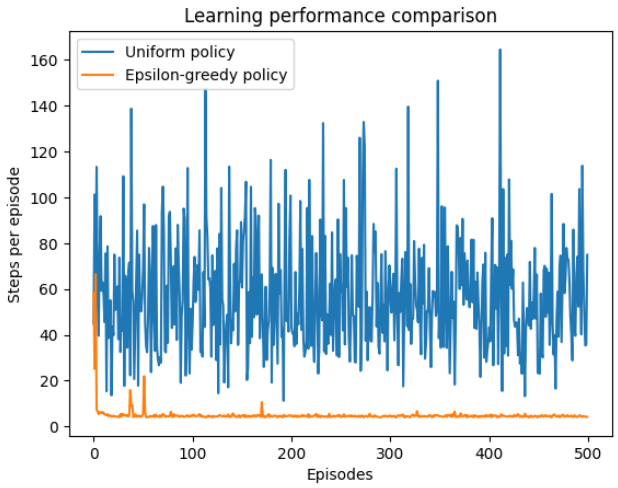
3. As of now, the off-policy Monte Carlo algorithm uses the equiprobable behavior policy to generate episodes. However, learning in such case cannot be fully taken advantage of, as we are ignoring the learned information in Q and keep exploring (uniformly). Instead, we can implement a variant of a ε-soft policy - the ε-greedy policy:

$$\mu(a \mid s) = \begin{cases} A_t = \arg\max\limits_{a \in \mathcal{A}} Q(s,a), & \text{with probability } 1-\varepsilon \\ \text{random } a \in \mathcal{A}, & \text{with probability } \varepsilon \end{cases}$$

This of course changes the importance sampling ratio:

$$W_n \longleftarrow W_n \left[\frac{1}{\mu(A_{n+1} \mid S_{t+1})} \right]$$

where μ is the behavior policy. With this, the agent utilizes what it has learned, while also maintaining a good level of exploration. I have additionally implemented breaking ties to avoid cycles (long episodes). The following plot is the number of steps per episode (averaged over multiple training runs) as a function of the number of episodes:



The above figure was computed using 10 runs where each algorithm, was generating 500 episodes. All formulas and algorithm implementations are based on the methods presented in Sutton & Barto.